

Berikut adalah penjelasan mendalam mengenai kode C# tersebut.

Ringkasan Fungsi

Kode ini mendefinisikan sebuah metode `TranslateMouseClickedToImagePoint`. Tujuan utamanya adalah **menerjemahkan posisi klik mouse pada PictureBox (UI) menjadi koordinat piksel yang sesuai pada gambar asli (Bitmap)**.

Hal ini sangat penting ketika PictureBox diatur ke mode **Zoom** (di mana gambar diperkecil/diperbesar agar muat di layar sambil mempertahankan aspek rasio), sehingga koordinat layar (X, Y) tidak sama dengan koordinat piksel gambar asli.

Penjelasan Per Baris / Blok Kode

1. Namespace dan Definisi Kelas

C#

```
using System.Drawing.Drawing2D;
using System.Windows.Forms;

namespace MiniPhotoshop.Logic.Helpers
{
    public static class CoordinateHelper
    {
```

- **Imports:** Mengimpor library standar untuk grafis dan UI Windows Forms.
- **CoordinateHelper:** Kelas ini dibuat static, artinya kita bisa memanggil fungsinya tanpa perlu membuat objek (`new CoordinateHelper()`). Ini adalah kelas utilitas.

2. Definisi Metode

C#

```
public static Point TranslateMouseClickedToImagePoint(PictureBox pb, Point mouseLocation)
```

- Menerima dua parameter:

1. pb: Kontrol PictureBox tempat gambar ditampilkan.
 2. mouseLocation: Koordinat X dan Y tempat mouse diklik pada layar (relatif terhadap pojok kiri atas PictureBox).
- Mengembalikan Point: Koordinat X dan Y pada file gambar asli.

3. Cek Keberadaan Gambar

C#

```
if (pb.Image == null)
{
    return mouseLocation;
}
```

- Jika tidak ada gambar yang dimuat di PictureBox, kembalikan saja lokasi mouse apa adanya (fail-safe).

4. Mengambil Dimensi

C#

```
float pWidth = pb.ClientSize.Width;
float pHeight = pb.ClientSize.Height;
```

```
float iWidth = pb.Image.Width;
float iHeight = pb.Image.Height;
```

- **pWidth/Height:** Lebar dan tinggi area tampilan (PictureBox).
- **iWidth/Height:** Lebar dan tinggi **asli** dari file gambar (resolusi asli).
- Menggunakan float agar perhitungan pembagian nanti presisi (memiliki koma desimal).

5. Menghitung Rasio Aspek

C#

```
float pRatio = pWidth / pHeight;
```

```
float iRatio = iWidth / iHeight;
```

- Menghitung perbandingan lebar terhadap tinggi.
- Ini digunakan untuk menentukan bagaimana gambar "pas" di dalam kotak (apakah ada ruang kosong di atas/bawah atau kiri/kanan).

6. Logika Penskalaan (Zoom Logic)

Blok ini mereplikasi logika `SizeMode.Zoom` milik Windows Forms untuk mencari tahu ukuran gambar yang sedang *ditampilkan* di layar dan posisi *offset*-nya (geserannya).

C#

```
float newWidth = iWidth;
```

```
float newHeight = iHeight;
```

```
float offsetX = 0;
```

```
float offsetY = 0;
```

```
if (pRatio > iRatio)
```

```
{
```

```
    newWidth = pHeight * iRatio;
```

```
    newHeight = pHeight;
```

```
    offsetX = (pWidth - newWidth) / 2;
```

```
}
```

```
else
```

```
{
```

```
    newWidth = pWidth;
```

```
    newHeight = pWidth / iRatio;
```

```
    offsetY = (pHeight - newHeight) / 2;
```

```
}
```

- **if (pRatio > iRatio):**
 - Artinya PictureBox lebih "lebar" (wide) daripada gambar aslinya.
 - **Konsekuensi:** Gambar akan mentok di atas dan bawah, tapi ada **ruang kosong (bar hitam/abu-abu) di kiri dan kanan**.
 - newHeight diset sama dengan tinggi PictureBox.
 - offsetX dihitung: (Lebar PictureBox - Lebar Gambar Baru) dibagi 2 (agar gambar ada di tengah).
- **else:**

- Artinya PictureBox lebih "tinggi" (tall) atau sama rasionya dengan gambar.
- **Konsekuensi:** Gambar mentok kiri dan kanan, ada **ruang kosong di atas dan bawah**.
- `newWidth` diset sama dengan lebar PictureBox.
- `offsetY` dihitung: (Tinggi PictureBox - Tinggi Gambar Baru) dibagi 2.

7. Rumus Translasi Koordinat

Ini adalah inti dari matematika konversinya:

C#

```
int imgX = (int)((mouseLocation.X - offsetX) * (iWidth / newWidth));
int imgY = (int)((mouseLocation.Y - offsetY) * (iHeight / newHeight));
```

Rumus ini bekerja dalam dua langkah:

1. **(mouseLocation.X - offsetX):** Menggeser titik nol. Jika ada ruang kosong di kiri sebesar 50px, dan kita klik di 60px, posisi sebenarnya pada gambar yang ditampilkan adalah 10px.
2. *** (iWidth / newWidth):** Faktor Skala. Karena gambar di layar (`newWidth`) mungkin lebih kecil dari gambar asli (`iWidth`), kita harus mengalikannya dengan rasio perbandingan (Scaling Factor) untuk mendapatkan koordinat piksel resolusi penuh.

$$PosisiAsli = (PosisiMouse - RuangKosong) \times \frac{UkuranAsli}{UkuranTampil}$$

8. Batasan (Clamping)

C#

```
if (imgX < 0) imgX = 0;
if (imgY < 0) imgY = 0;
if (imgX >= iWidth) imgX = (int)iWidth - 1;
if (imgY >= iHeight) imgY = (int)iHeight - 1;
```

- Kode ini mencegah *Index Out of Bounds*.
- Jika pengguna mengklik di area "ruang kosong" (bar hitam di pinggir gambar), hasil perhitungan matematika di atas bisa menjadi negatif atau lebih besar dari ukuran

- gambar.
- Kode ini memaksa (clamp) nilai agar tetap berada dalam batas 0 sampai (Lebar/Tinggi - 1).

9. Pengembalian Nilai

C#

```
return new Point(imgX, imgY);
```

- Mengembalikan koordinat X,Y yang sudah valid pada gambar asli.

Contoh Kasus Penggunaan

Bayangkan Anda membuat aplikasi "Mini Photoshop":

1. Gambar asli berukuran **1000x1000 pixel**.
2. Layar PictureBox berukuran **500x500 pixel**.
3. Gambar ditampilkan dalam mode Zoom (terlihat penuh).
4. Pengguna mengklik tepat di tengah layar (**250, 250**).

Tanpa kode ini, aplikasi akan mengira pengguna mengklik piksel ke-250.

Dengan kode ini:

1. Offset dihitung (0, karena aspek rasio sama).
2. Skala dihitung: $1000 / 500 = 2$.
3. $\text{imgX} = 250 * 2 = \mathbf{500}$.
4. Hasilnya adalah titik tengah pada gambar resolusi tinggi tersebut.

Apakah Anda ingin saya menjelaskan bagaimana cara menggunakan helper ini di event `MouseClicked` pada Form Anda?